DEPT: Defense via Ensemble and Purification Transformer

Extended Abstract of the MSc Thesis in Computer Science and Engineering

GUILHERME SILVÉRIO DE CARVALHO ROMEIRO LEITÃO, Instituto Superior Técnico, Universidade de Lisboa, Portugal

Machine-learning-based network intrusion detectors can be evaded by an attacker that reshapes only packet timing and sizes, keeping the traffic protocol-compliant and the attack functional. Much of the defensive literature measures robustness against feature perturbations without checking whether they correspond to sendable packet sequences. We present DEPT (Defense via Ensemble and Purification Transformer), a streaming defense in front of a protected detector. An attention-based meta-learner fuses, at every packet, a Random Forest over flow statistics and a Temporal Convolutional Network over masked bit-level packet headers to flag manipulated flows; a causal Transformer corrects flagged flows in feature space, and the corrections are written back onto packet timing and size. DEPT is trained and evaluated on realizable adversarial traffic from the black-box generator TANTGen, over ten traffic sources from three public datasets and three perturbation modes. It predicts from the first packet, reaching an end-of-flow AUROC of 0.933. The Random Forest alone has a higher AUROC from the second packet on, but against a gradient-guided attacker the ensemble is the hardest target: 74.5% of flows never evade it, against 45.5% for the network alone. Judged by the independent detector each attack was built to evade, purification recovers 63% of the lost detection, the full pipeline cuts attack success from 21.1% to 11.6%, and a published diffusion-based purifier in the same position recovers almost none. Among realizable packets, 97.2% parse without errors. Detection stays near or below chance on size-perturbed reflection floods, and purification can lower detection where the manipulation had not evaded.

CCS Concepts: • **Security and privacy** → **Intrusion detection systems**; *Network security*; • **Computing methodologies** → *Ensemble methods*; Neural networks.

Additional Key Words and Phrases: Adversarial machine learning, network intrusion detection, evasion attacks, ensemble learning, adversarial purification, streaming traffic analysis

1 Introduction

Machine learning has become a common approach to network intrusion detection. Instead of matching known signatures, a learned network intrusion detection system (NIDS) models the statistical behaviour that separates malicious from benign traffic, and can therefore generalize to attack variants that no signature describes [Boutaba et al. 2018; Wang et al. 2021]. It also brings a new attack surface. A small, deliberate change to a model's input, an *adversarial example* [Goodfellow et al. 2014; Szegedy et al. 2014], can make the model misclassify. For a NIDS this means *evasion*: the attacker reshapes ongoing attack traffic just enough for the detector to label it benign, while the attack keeps working [He et al. 2023]. Black-box attacks of this kind reached evasion rates of up to 35.7% against deep-learning detectors that looked nearly perfect on clean benchmarks [Zhang et al. 2020].

Two constraints separate this setting from the image domain, where adversarial examples were first studied. First, a perturbed feature vector of traffic is a threat only if some packet sequence produces it: packets must keep their checksums, lengths and protocol semantics consistent, and recovering valid packets from an arbitrary feature vector is often intractable [Hashemi et al. 2019; Pierazzi et al. 2020]. Second, a realistic attacker is black-box: it can probe a deployed detector and observe the outcome, but has no access to the model's architecture or gradients [Apruzzese et al. 2023; Papernot et al. 2017]. Much of the defensive literature sets aside one or both constraints, so its robustness results may not hold against threats that are actually feasible.

This work is the defense side of an effort shared with a companion thesis. On the attack side, TANTGen [Pissarra 2026] is a Transformer-based generator that manipulates attack traffic under black-box constraints. It changes only packet timing and size, and the traffic it emits is protocol-compliant by construction. We propose DEPT (Defense via Ensemble and Purification Transformer), a streaming defense placed inline in front of a protected NIDS (Figure 1). DEPT decides, packet by packet, whether a flow has been adversarially manipulated and, when it has, tries to undo the manipulation before the protected NIDS processes the flow. It is trained and evaluated on TANTGen's traffic, but does not depend on TANTGen alone.

Three requirements shape the design. The verdict must come while the flow is still in progress (*streaming operation*), since waiting for a flow to end may let the attack succeed first. The defense must be trained and evaluated against protocol-compliant traffic generated under black-box assumptions (*realistic attacks*). And a flow the defense repairs must remain valid traffic (*realizable output*); otherwise the defense only moves the realizability problem from the attacker to the defender. DEPT complements the protected NIDS and does not replace it: recognizing the underlying attack stays with that detector. Two difficulties come from the data. The traffic used in this work is dominated by high-rate floods, so aggregate results can hide poor performance on the other sources, and a manipulation confined to timing may change the observed features very little.

Two decisions follow from these requirements. For detection, DEPT stacks two learners from different architectural families: a Random Forest (RF) over flow statistics, whose decision function offers no gradient, and a Temporal Convolutional Network (TCN) over bit-level packet sequences. A small attention-based meta-learner fuses them at every packet [Wolpert 1992]. A black-box attacker relies on transferability, crafting a perturbation against a substitute model and hoping it carries over to the target [Papernot et al. 2017]; a perturbation that moves one learner does not necessarily move the other. For a flagged flow, dropping it would turn every false positive into a denial of service, so DEPT purifies it instead [Samangouei et al. 2018]: a causal Transformer predicts a residual correction for each packet from the packets before it, and a realization step writes the corrected sizes and timings back onto real packets. The contributions are:

- a packet-to-tensor pipeline that turns raw captures into identity-masked, bit-level packet tokens and incremental flow statistics, so that two architecturally different learners read the same flow at different granularities (Section 4.2);
- a heterogeneous detection ensemble whose meta-learner fuses a dense and a sparse prediction stream into a per-packet verdict, evaluated by how early, and not only how well, it detects manipulation (Sections 4.3 and 5.2), and against a gradient-guided attacker (Section 5.3);
- a causal purification Transformer coupled to an explicit realization step, whose packets are checked with a standard protocol dissector (Sections 4.4 and 5.4);
- an end-to-end evaluation of DEPT in front of the independent NIDS each attack was built to evade, including a comparison with the published diffusion-based purifier of Merzouk et al. [2025] (Section 5.5);
- an adversarial corpus of ten traffic sources from three public datasets [Mirsky et al. 2018; Sharafaldin et al. 2018, 2019], each manipulated by TANTGen in three modes, with a per-source analysis of where the defense works, where it fails, and why (Section 5).

DEPT gives a verdict from the first packet, and against a gradient-guided attacker three quarters of the flows never evade the ensemble at any budget we tested. Through the independent detectors, purification recovers 63% of the detection lost to TANTGen, and the full pipeline cuts attack success from 21.1% to 11.6%, while the diffusion purifier recovers almost none. The failures are reported with the same weight: size-perturbed reflection floods remain undetected, purification can harm detection where the attack had not evaded, and the meta-learner falls below its own Random Forest after the first packet.

2 Background and Related Work

2.1 Evasion of Learned Network Detectors

Attacks on learned models are usually classified by what the attacker knows (white-box, gray-box or black-box access) and by what it wants: poisoning corrupts the model during training, while evasion manipulates inputs at inference time [He et al. 2023]. Black-box evasion is the realistic case for a deployed NIDS, and it usually relies on transferability [Papernot et al. 2017]. In networking, a second distinction matters: a perturbation can be expressed in the *feature space* of vectors derived from traffic, or in the *traffic space* of packets on the wire. The first attacks applied gradient methods from the image domain, such as the Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD) [Goodfellow et al. 2014; Madry et al. 2018], to NIDS feature vectors [Hashemi et al. 2019; Zhang et al. 2020]. Their evasion rates are upper bounds obtained in a space the attacker does not control, since finding a valid packet for a perturbed vector, the inverse feature mapping problem, is often constrained or intractable [Pierazzi et al. 2020]. Recent attacks therefore work in the traffic space and enforce realizability by verification or by construction. GPMT generates practical malicious traffic with a generative adversarial network (GAN) [Sun et al. 2023]; PANTS, a white-box tool, uses a satisfiability modulo theories (SMT) solver to turn each adversarial input into a nearby, valid packet sequence [Jin and Apostolaki 2025]; DeepRed embeds a GAN-based generator in a command-and-control framework and lowers the detection rate of ML-based NIDS by up to 20% [Hajizadeh et al. 2025]; and NetDiffuser brings diffusion models to the attack side, although under a white-box assumption [Kumar et al. 2026].

TANTGen. TANTGen [Pissarra 2026] uses self-attention to model the dependencies between the packets of a flow and generates its evasions directly in the traffic space, so protocol compliance holds by construction. It is black-box with respect to its target: the gradients that guide it come from a surrogate NIDS of its own, trained on labelled traffic. It changes only two quantities of live attack traffic, packet timing (inter-arrival times) and packet size (payload padding), in three perturbation modes: `iat`, `size` and `both`. Its evasion was calibrated against Kitsune [Mirsky et al. 2018], an anomaly detector built from an ensemble of autoencoders, for the Kitsune attacks, and against its own flow classifiers (ML-NIDS) for the CIC attacks. These properties, and not TANTGen itself, have three consequences for a defense. Checking protocol validity is useless, so any signal must come from the traffic's timing and sizing behaviour; the footprint of the manipulation can be thin, mainly in the timing-only mode; and transferability is the attacker's main channel.

2.2 Defenses against Evasion

Training-time defenses. Data sanitization [Rubinstein et al. 2009] and federated backdoor detection [Rieger et al. 2024] protect the training pipeline against poisoning, a threat that DEPT excludes. Bagging also limits poisoning [Biggio et al. 2011; Chen et al. 2022], and shows from the training side that ensemble diversity is a robustness mechanism in itself. Ensemble hardening builds this diversity into the detector. Tiki-Taka proposes model voting and ensemble adversarial training [Zhang et al. 2020], TRS trains ensemble members for gradient diversity and smoothness, which lowers transferability between them [Yang et al. 2021], Stewart shows, in predictive maintenance, that a stacking ensemble withstands white-box attacks better than its members [Gungor et al. 2022], and Mateen keeps an adaptive committee of autoencoders for anomaly detection under distribution shift [Alotaibi and Maffeis 2024]. Their members almost always come from the same differentiable family, so every member exposes a gradient and protection rests on decorrelating those gradients. Adversarial training [Madry et al. 2018] is only meaningful in networking when the adversarial examples are realizable; PANTS adds its realizable examples to the training loop and obtains robustness that holds against attacks not seen in training [Jin and Apostolaki 2025].

Inference-time defenses. Certified defenses guarantee that no perturbation within a bounded region changes a prediction: CertTA uses randomized smoothing over packet timing, sizing and insertion [Yan et al. 2025], and LipsLev certifies text classifiers under a bounded Levenshtein distance [Abad Rocamora et al. 2025]; both also train or fine-tune the model for the certificate. A timing or size manipulation, however, is not bound to a radius, since a flow's timing can be stretched by large amounts and remain realizable. *Purification* transforms each input before classification to remove the perturbation, projecting it back onto the manifold of normal

Table 1. Surveyed defenses and DEPT, by the phase at which they act and the space they operate in. T: the defense changes the protected model; I: it acts on each input or prediction and leaves the model unchanged. †Outside the network domain.

System	Approach	Phase	Space
Antidote	Robust-statistics sanitization	T	feature
Tiki-Taka	Voting, ensemble adv. training	T	feature
TRS†	Gradient-diverse ensemble	T	feature
PANTS	Realizable adv. training	T	traffic
CertTA	Randomized smoothing	T, I	traffic
Defense-GAN†	GAN projection	I	feature
AdvPurRec	Diffusion purification	I	feature
Merzouk et al.	Diffusion purification	I	feature
NIDS-CBAD	Domain-constraint checks	I	feature
AMDS	Fused ensemble signals	I	feature
DEPT	Stacking, realized purification	I	traffic

traffic. The idea goes back to Defense-GAN [Samangouei et al. 2018] and has reached the network domain with diffusion models, in AdvPurRec [Alhussien and Aleroud 2024] and in the work of Merzouk et al. [2025], who map the trade-off between removing more perturbation with more noise and losing clean accuracy. Other detectors flag violations of inter-feature constraints, as in NIDS-CBAD [Kumar et al. 2025], which protocol-compliant traffic does not produce, or fuse ensemble disagreement and uncertainty, as in AMDS [Olukola and Rahimi 2026]. Finally, a model without a usable gradient is a case of gradient masking, which can give a false sense of security [Athalye et al. 2018]; the value of DEPT's non-differentiable member is therefore measured (Section 5.3), not assumed.

2.3 The Gap

Table 1 summarizes the surveyed defenses. In the feature space, every kind of defense has examples. In the traffic space, PANTS hardens the model before deployment and CertTA certifies predictions within a bound, but no surveyed defense takes a flow as it arrives, decides whether it was manipulated and returns it with the manipulation undone. The network purifiers do this only on feature vectors, so their output serves only a detector that reads those features. DEPT takes this position: it works at inference time, in front of a NIDS that it leaves unchanged, and returns valid packets. Its other choices follow from it. Edited traffic must leave as valid packets, hence the realization step; a learned detector in front of the NIDS can itself be attacked through transfer, hence the heterogeneous ensemble; and both stages need examples of the manipulation, hence training on a generator's traffic, which fits DEPT's own models and does not retrain the protected NIDS. Its purifier also differs from those in the Defense-GAN tradition in two ways. It learns from matched adversarial-to-clean flow pairs, a stronger supervision that a paired generator such as TANTGen makes available, instead of from clean data alone. And it corrects each packet from the preceding ones in a single causal pass, whereas a diffusion purifier needs the whole input before it can denoise it.

3 Threat Model

We distinguish *benign* traffic (no attack), *attack* traffic (a conventional attack, such as a flood or a scan, that a NIDS is built to detect) and *adversarial* traffic (attack traffic perturbed so that the NIDS no longer detects it). Three parties are involved (Figure 1). A *protected NIDS* guards a network and is assumed to handle benign and attack traffic correctly, but not adversarial traffic. An *attacker* produces adversarial traffic by manipulating an ongoing attack. *DEPT* sits inline, immediately upstream of the protected NIDS: it must decide, packet by packet, whether a flow is adversarial, and neutralize the manipulation before the NIDS consumes the flow.

The attacker is any adversary with the capabilities below, here TANTGen, so they are demonstrated rather than assumed. It has no access to the architecture, parameters or gradients of DEPT or of the protected NIDS, and can at most observe detection outcomes. It perturbs inter-arrival times and payload padding, and its packets are protocol-compliant. Attacks that break protocol semantics, such as a wrong checksum, an inconsistent length field or a negative inter-arrival time, are out of scope: they are not working evasions, and stateless checks reject them before any learned model is involved.

TANTGen never takes gradients from DEPT, so it cannot test the claim behind the ensemble, which is a claim about gradient paths. A second adversary is therefore used for the first research question (Section 5.3). It has white-box access to the TCN (architecture, weights and gradients) and nothing for the Random Forest, which has no gradient to leak. Its perturbations are protocol-compliant and *increase-only*, since packets on the wire can be padded or delayed but not un-sent, and its budget is counted in the units an attacker spends: bytes of padding and seconds of delay.

DEPT, in turn, does not use TANTGen's model, parameters or surrogate NIDS, only its traffic: labelled flows train the detectors, and flows paired with their clean originals train the purifier, so any generator with the capabilities above could replace TANTGen. It is not black-box all the way through, however, since realization writes back only the two quantities the attacker perturbs (Section 4.4): DEPT knows the attacker's feature set but not its model, a gray-box position. Training and test flows come from the same generator, so DEPT is evaluated against the attacker it was trained on; only the second adversary's added perturbations are never seen in training.

Three non-goals complete the model. DEPT does not defend against poisoning: the training data comes from known sources and the training pipeline is trusted. No claim is made against an adversary with gradient access to every component, which for the Random Forest is not possible. And DEPT does not detect the underlying attacks (the flood, the scan): it only answers whether a flow has been adversarially manipulated and undoes the manipulation when it has.

4 DEPT Architecture

4.1 Overview

Figure 1 shows the architecture. A packet-to-tensor pipeline derives two views of each flow: a dense stream of masked bit-level tokens, one per packet, and a sparse stream of statistics over the flow's prefix. The TCN reads the dense stream and emits a logit at every packet, from the first one; the Random Forest reads the sparse stream and

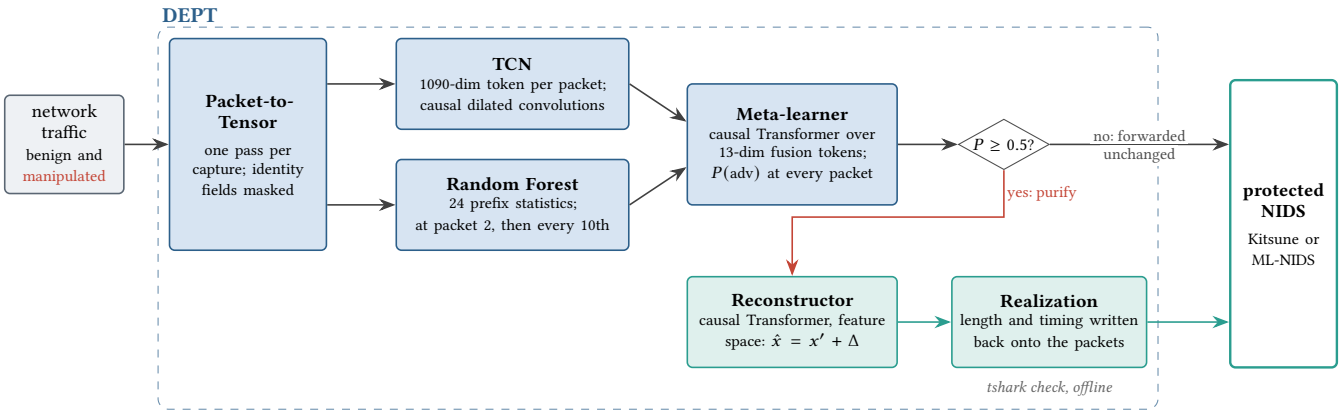


Fig. 1. DEPT in front of a protected NIDS. The packet-to-tensor pipeline derives two views of each flow: bit-level packet tokens for the TCN and prefix statistics for the Random Forest. The meta-learner fuses the two prediction streams into a verdict at every packet. Unflagged flows pass unchanged; flagged flows are corrected by the reconstructor and realized back into packets, so the NIDS receives traffic, not tensors.

emits a logit at packet 2 and then every tenth packet. A per-packet *fusion token* joins the two prediction streams, and the meta-learner, a small causal Transformer, turns it into a per-packet probability that the flow is adversarial. Flows that never cross the threshold are forwarded unchanged, so unflagged traffic pays for detection but not for purification. Flagged flows go to the *reconstructor*, which corrects them in feature space, and a *realization* step writes the correction onto the packets. All components are trained offline in dependency order: the two base learners on the training split, the meta-learner on the validation split, and the reconstructor on matched pairs of adversarial and clean flows.

4.2 Corpus and Traffic Representation

Corpus. The corpus pairs ten traffic sources with their TANTGen counterparts: the Mirai botnet and SSDP flood captures of Kitsune [Mirsky et al. 2018]; the DDoS, port scan and FTP/SSH brute-force captures of CIC-IDS2017 [Sharafaldin et al. 2018]; and the NTP, DNS, SSDP and LDAP reflection and amplification floods and a SYN flood of CIC-DDoS-2019 [Sharafaldin et al. 2019]. These floods offer no clean time window to filter on, so each is truncated to its first million packets, the scale at which TANTGen caps its own input. TANTGen produced one variant per perturbation mode, giving 30 adversarial source–mode combinations; tensorized, the 40 sources hold 8,624,958 flows and 61,516,406 packets.

Labels. Every flow of the raw datasets is labelled *benign*, including their native attacks, and every TANTGen flow *adversarial*. Detecting a flood or a scan is the protected NIDS's task; DEPT must recognize whether the generator touched the flow. In the rest of this paper, *benign* therefore means *unmanipulated*. The cost of this convention is that, where TANTGen barely changes a flow, the two classes are nearly indistinguishable for any detector.

Packet tokens. Each capture is processed in one pass, grouping packets into flows by their five-tuple and keeping them in time order. Following nPrint [Holland et al. 2021], the IPv4, TCP, UDP and ICMP headers of each packet are encoded bit by bit into 1,088 tri-state cells (1 or 0 for a set or unset bit, −1 for a layer absent from the packet), to which we append the z-scored inter-packet arrival time (IPAT) and a direction bit, giving a 1,090-dimensional token. The 144 bits that identify hosts and ports (IP addresses, transport ports and IP identification) are set to zero, so that a model cannot score well by memorizing the hosts of a capture [Arp et al. 2022] and must rely on behaviour, such as sizing and timing.

Prefix statistics. For the Random Forest, the same pass measures six quantities on the packets themselves: IPAT, packet size, payload length, payload entropy, TCP flags and direction. Each is summarized by its mean, standard deviation, minimum and maximum over the prefix seen so far, a 24-dimensional vector updated incrementally (Welford's algorithm and running extremes), so the cost per packet is constant. Checkpoints fire at packet 2 and then every tenth packet. The early checkpoint exists because some attacks are very short: a port scan averages about two packets per flow. Since nPrint encodes headers only, the payload entropy that the forest uses is invisible to the TCN, which adds to the diversity between the two learners.

Split and class balance. All models share one 70/15/15 split into training, validation and test sets, made at flow level (every token, checkpoint or pair of a flow falls on the same side) and stratified by source. Their results are therefore directly comparable, and the meta-learner is never evaluated on flows that a base learner trained on. Adversarial flows outnumber benign ones about five to one, so the forest and the neural losses use balanced class weights.

4.3 Heterogeneous Detection Ensemble

Random Forest. At every checkpoint, the RF [Breiman 2001] decides whether the prefix statistics look manipulated. Forests are strong and cheap on small tabular inputs, and their decision function is piecewise constant, with no informative gradient for an attacker to follow. A grid search on the validation split selected 200 trees of maximum depth 20 with at least five samples per leaf, the most regularized setting of the grid.

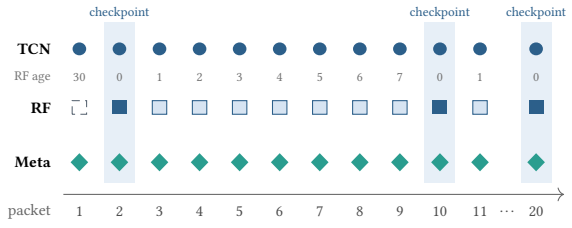


Fig. 2. Prediction cadence over the first packets of a flow. The TCN emits a logit at every packet; the RF emits one at packet 2 and every tenth packet (dark squares) and its last logit is carried forward (light squares), with its age in packets given to the meta-learner, which fuses both at every packet.

TCN. The TCN [Bai et al. 2018] reads the token stream and emits a logit at every packet. It has three residual blocks of 128 channels, each with two causal convolutions of kernel size $k = 3$, with dilations 1, 2 and 4 (about 0.8M parameters). Its receptive field is $1 + 2(k-1)(1+2+4) = 29$ packets, so a verdict depends on the current packet and the 28 before it, and never on future packets. The per-packet cross-entropy is averaged within each flow before averaging across flows, so that a long flood does not drown two-packet scans in the gradient.

Meta-learner. The difficulty in fusing the two learners is asynchrony (Figure 2): at most packets the forest has no fresh opinion, and before packet 2 it has none, so a fixed rule such as averaging is not even defined everywhere. The meta-learner is a decoder-only Transformer [Vaswani et al. 2017] (two causal blocks, four heads, $d_{\text{model}} = 64$, about 68k parameters) over a 13-dimensional fusion token: the RF logit carried forward from the last checkpoint (a learnable placeholder before packet 2), the TCN logit, the number of packets since the last RF refresh, a flag marking a refresh at this packet, and a small summary of the packet (z-scored IPAT, direction, length and six TCP flags). The full header representation is withheld on purpose, so that the model combines its base learners instead of bypassing them. To avoid learning the overfit confidence of the base learners [Wolpert 1992], it is trained on the validation split, where their predictions are out-of-fold. Its loss is the same balanced per-packet cross-entropy plus an auxiliary term (weight 0.5) that penalizes, at each packet, any shortfall of its confidence in the true class against the better of the two base learners. A flow is flagged the first time its probability p_t crosses 0.5.

4.4 Purification and Realization

Reconstructor. The reconstructor is a causal Transformer denoiser over the token sequence (three layers, four heads, $d_{\text{model}} = 128$, about 678k parameters). Given an adversarial flow $x' = (x'_1, \ldots, x'_L)$, it predicts a residual correction for every token and outputs

$$\hat{x}_t = x'_t + \Delta_t, \qquad \Delta_t = g_\theta(x'_1, \ldots, x'_t), \tag{1}$$

so each correction depends only on the current and earlier packets. The attacker moves only a few of the 1,090 dimensions of a token, so with a residual head the identity ($\Delta \approx 0$) is the easy default, and the roughly 1,050 untouched header bits are preserved. The model is not autoregressive: it corrects all positions in one pass, and the causal mask is there for streaming. Unlike generative purifiers, it does not model the distribution of benign traffic; it regresses the correction directly.

Table 2. Configuration of the four learned components. A step is a checkpoint for the RF and a packet for the three sequence models.

	RF	TCN	Meta-learner	Reconstructor
Input per step	24	1,090	13	1,090
Structure	200 trees	3 blocks	2 layers	3 layers
Width	depth 20	128 channels	$d = 64$, 4 heads	$d = 128$, 4 heads
Regularization	min. leaf 5	dropout 0.15	dropout 0.1	dropout 0.1
Parameters	—	805,762	67,971	677,698
Loss	Gini	balanced CE	bal. CE + aux.	smooth-L1
Learning rate	—	10^{-3}	$3 \cdot 10^{-4}$	$3 \cdot 10^{-4}$
Batch size	—	32	128	32
Selection	val. macro-F_1	val. macro-F_1	AUROC@10	val. loss

Training on matched pairs. The attacker keeps a flow's five-tuple, so an adversarial flow can be aligned with the clean flow it was derived from. The reconstructor is trained on these (adversarial → clean) pairs, without single-packet pairs, using a smooth-L1 loss between $\hat{x}$ and the clean flow, averaged within each flow. The loss weights all 1,090 dimensions equally, so the training signal holds no hint of where the perturbation is: the model has to find which dimensions were tampered with. This keeps the feature-space stage attack-agnostic.

Realization. Packets cannot be rebuilt from the purified tensors alone, since nPrint encodes headers only and the identity fields are masked. Realization therefore works by overlay: each original packet keeps its identity and payload content, and only the two quantities purification may change are written back. The purified IPv4 total length, decoded from its 16 header bits, is applied by trimming the payload, the inverse of the attacker's padding, and the purified IPAT is de-normalized and applied to the packet's timestamp. Packets are then re-serialized with scapy, which recomputes checksums and length fields. The price is that the deployed pipeline is not attack-agnostic end to end: a flow manipulated in another field, such as the time-to-live (TTL), would be flagged and denoised in feature space, but would reach the NIDS unrepaired. Every realized packet is finally checked with the `tshark` dissector; this check is a measurement made offline, not a step on the inference path. A size edit on an application-layer (L7) text protocol, such as the SSDP `M-SEARCH` messages of one source, breaks the L7 grammar even when the IP and UDP fields stay valid. Such packets cannot be realized faithfully by overlay and are reported separately.

4.5 Implementation

Table 2 summarizes the learned components; the sequence models are trained with AdamW for at most 20 epochs, with early stopping on the selection criterion of the table. DEPT is written in Python, with PyTorch [Paszke et al. 2019] for the three sequence models, scikit-learn [Pedregosa et al. 2011] for the forest, and a Python reimplementation of the nPrint encoding on top of scapy, which also performs realization; `tshark`, `editcap` and `tcpdump` handle dissection, time filtering and packet capping. All experiments ran on one server with two Intel Xeon Gold 5320 CPUs, 160 GB of RAM

and two NVIDIA RTX 6000 Ada GPUs. Generating the 30 adversarial captures took 26.7 h and tensorization 11.2 h; training took 2.7 h for the RF (with its grid search), 24.7 h for the TCN, 3.4 h for the meta-learner and 10 h for the reconstructor.

5 Evaluation

5.1 Research Questions and Setup

The evaluation answers two research questions. **RQ1** (ensemble robustness): does heterogeneous stacking measurably raise the cost of a transferable, gradient-guided evasion attack, compared with a single-architecture model? **RQ2** (purification): does the reconstructor bring a flagged flow back towards its clean form, closing the detection gap seen by the protected NIDS, while emitting realizable packets? Before them, we report how early and how well DEPT detects manipulation. The latency of the full pipeline is not evaluated in this work.

Test data. The held-out test split has 1,293,737 flows, of which 83.7% are adversarial, because the CIC-DDoS-2019 floods dominate; the RF, whose first checkpoint is packet 2, does not score the 6,179 single-packet flows. Because of this base rate, the models are compared with the area under the ROC curve (AUROC) and on the streaming axis rather than on accuracy. Of the 7,217,272 adversarial flows, 2,566,489 have a matched clean counterpart, of which 1,796,542 pairs are used for training and 384,963 are held out for testing.

Metrics. AUROC@t is the AUROC computed with each flow's prediction after its first t packets. The earliest detection packet (EDP) is the median packet at which a flow's score first crosses 0.5, and the detection rate is the fraction of adversarial flows that ever cross it. For purification, the reconstruction MSE is computed on the dimensions TANTGen perturbed, between a flow and its true clean counterpart; as cells are ternary, 1.0 is the typical cost of a flipped bit. The main purification metrics are measured through an *independent* NIDS: the traffic of each attack is scored by the detector TANTGen calibrated that attack against, Kitsune (KitNET with AfterImage) for the two Kitsune sources and TANTGen's per-attack CICFlowMeter flow classifiers (ML-NIDS) for the eight CIC attacks, so no DEPT component grades itself. With D the detection rate of that NIDS, we report the attack success rate ASR $= 1 - D$ and the restoration ratio

$$\rho = \frac{D_{\text{pur}} - D_{\text{adv}}}{D_{\text{clean}} - D_{\text{adv}}}, \tag{2}$$

the fraction of the detection lost to TANTGen that purification recovers. A gain measured through DEPT's own TCN is kept only as a sanity check, since it is circular. Protocol compliance is the fraction of realized packets that `tshark` parses without a malformed-packet error.

5.2 Detection

Table 3 compares the three models on the streaming axis. The Random Forest alone is the best model from packet 2 on, at 0.9805 at its first checkpoint and 0.9813 at the end, against 0.9099–0.9334 for the meta-learner and 0.8225–0.8386 for the TCN. We expected the meta-learner to be at least as good as its best base learner, and it is not. It is the best model only at packet 1, where the forest has no verdict yet (0.8373 against 0.8191 for the TCN), and from packet 2 on it stays 8.7 to 9.5 points above the TCN. Its main contribution is robustness: at low budgets it is the hardest of the three models to evade (Section 5.3). At the 0.5 threshold it is conservative, flagging only 1,115 of the 211,151 benign test flows (precision 0.9985).

Table 3. Streaming detection on the held-out test flows. AUROC@t uses each flow's prediction after t packets; EDP is the median packet of first detection; the detection rate counts adversarial flows ever flagged.

Metric	RF	TCN	Meta-learner
First verdict at	packet 2	**packet 1**	**packet 1**
AUROC@1	—	0.8191	**0.8373**
AUROC@2	**0.9805**	0.8225	0.9099
AUROC@5	**0.9805**	0.8375	0.9319
AUROC@10	**0.9813**	0.8371	0.9323
AUROC@max	**0.9813**	0.8386	0.9334
EDP (median)	2	**1**	**1**
Detection rate	**88.99%**	78.40%	77.69%

The aggregate hides large differences between sources. Table 4 measures each source–mode combination against benign flows of the same source, since DEPT must tell a manipulated flow from an unmanipulated flow of the same kind. Most combinations are solved: 21 of the 30 reach at least 0.99 for the RF, and 13 for the meta-learner. Timing-only edits are hard for the TCN and easy for the forest (`portscan iat`: 0.652 against 1.000; `syn_ddos iat`: 0.647 against 1.000), because the forest's timing statistics capture a change in timing that the per-packet view misses. This is the kind of difference between learners that the ensemble was built on. The TCN fails on the four reflection floods, at 0.494–0.552 in every mode: their flows are one- or two-packet UDP exchanges, and a receptive field of 29 packets has nothing to convolve over. The `size` variants of the same floods are at or below chance for every model, so there a TANTGen flow looks more like generic benign traffic than the flood's own unedited flows do (Section 6). Finally, no model solves `mirai` in any mode, but none falls below chance on it either.

On some sources the meta-learner falls far below the forest: on `ntp_ddos both` and `iat` it reaches 0.653–0.654 where the forest reaches 0.999, and on `ssdp_ddos` 0.927 against 1.000. A likely cause is that the RF logit enters the fusion token as a single feature refreshed only at checkpoints, while the TCN logit changes at every packet, so attention gives the forest too little weight. The auxiliary loss falls from 0.0267 to 0.0177 during training, moving the meta-learner towards its better base learner, but the forest stays ahead.

Per-attack specialists. Detectors in the literature are often trained per attack, so we also ran the four model stages once per attack, with the same architectures, searches and split protocol. The specialist and global meta-learners are indistinguishable on 13 of the 30 source–mode combinations. Specialists are better on timing edits: `ntp_ddos both` and `iat` (+0.346 and +0.347 AUROC), `syn_ddos iat` (+0.194), `ssdp_ddos both` and `iat` (+0.073) and `portscan iat` (+0.066), since a model trained on one attack can learn that source's own timing. The global model is better on `mirai both` and `iat` (by 0.035 and 0.033) and on the `size` variants of the reflection floods, where both are below chance and the global model is only less wrong

Table 4. End-of-flow AUROC per source and perturbation mode, against benign flows of the same source. n_{adv} is the number of adversarial test flows per mode (4,842 on ddos size; on mirai, the RF scores 867 of the 1,957, as many flows end before packet 2, and on ssdp_flood 9,830). Values below 0.55 are marked with †.

		both			iat			size		
Source (dataset)	n_{adv}	RF	TCN	Meta	RF	TCN	Meta	RF	TCN	Meta
brute_force (CIC-IDS2017)	1,045	0.974	0.978	0.981	0.974	0.978	0.979	0.990	0.978	0.989
ddos (CIC-IDS2017)	5,781	1.000	1.000	1.000	0.999	0.924	0.967	1.000	1.000	1.000
portscan (CIC-IDS2017)	23,800	1.000	0.997	0.998	1.000	0.652	0.757	0.998	0.997	0.997
mirai (Kitsune)	1,957	0.926	0.680	0.724	0.919	0.680	0.719	0.829	0.723	0.734
ssdp_flood (Kitsune)	9,831	1.000	0.999	1.000	1.000	1.000	1.000	1.000	0.999	1.000
syn_ddos (CIC-DDoS-2019)	200,483	1.000	1.000	1.000	1.000	0.647	0.806	1.000	1.000	1.000
dns_ddos (CIC-DDoS-2019)	1,019	0.999	0.552	1.000	0.999	0.547†	1.000	0.258†	0.531†	0.443†
ldap_ddos (CIC-DDoS-2019)	74,710	1.000	0.500†	1.000	1.000	0.500†	1.000	0.476†	0.501†	0.476†
ntp_ddos (CIC-DDoS-2019)	2,103	0.999	0.514†	0.654	0.999	0.497†	0.653	0.213†	0.494†	0.493†
ssdp_ddos (CIC-DDoS-2019)	40,446	1.000	0.499†	0.927	1.000	0.502†	0.927	0.409†	0.497†	0.497†

(0.443 against 0.192 on dns_ddos). The meta-learner also falls below its own forest in the specialists (on mirai, 0.705 against 0.889), and the per-attack TCNs also fail on the reflection floods (0.500–0.531) while doing well on longer flows (0.990 on ddos), so neither problem comes from training across sources. A global model remains the better default, because a specialist needs a router that names the attack first, which is the classification problem the detector is there to solve. Specialists are only worth considering for a known flood whose evasion is in the timing.

5.3 Robustness to a Gradient-guided Attacker (RQ1)

Attack. TANTGen never takes gradients from DEPT, so it cannot answer RQ1. The second adversary of Section 3 crafts one perturbation against the TCN with PGD [Madry et al. 2018] and replays it, unchanged, through the TCN, the RF and the fused stack; re-optimizing it for each target would no longer measure transfer. The perturbation adds padding (random bytes) and delay to each packet, on top of TANTGen's own, under five per-packet budgets from 8 bytes and 10 μs to 1,460 bytes and 100 ms. Three controls guard against a weak attack [Carlini et al. 2019; Tramèr et al. 2020]: a random search with the same feasible set and number of queries, a cross-check with the parameter-free AutoPGD [Croce and Hein 2020] of the ART library [Nicolae et al. 2018], which agrees with our PGD, and a reported attacker that takes the best of PGD and random search for each flow. We sample 9,000 test flows (300 per source–mode combination), set each target's threshold at a matched 5% false-positive rate on benign traffic, and measure ASR only over the flows each target detects without perturbation (7,510 for the RF, 3,374 for the TCN and 7,597 for the stack).

Results. Figure 3 shows the outcome. The perturbation breaks the TCN it was crafted against and transfers poorly to the forest, and at low budgets the fused stack is harder still: at 8, 32 and 128 bytes per packet it concedes 1.7%, 4.7% and 8.5% attack success, against 8.5%, 10.7% and 13.1% for the RF and 40.5% to 52.2% for the TCN. Over the whole grid, 74.5% of the flows never evade the stack, against 64.0% for the RF and 45.5% for the TCN. At 512 bytes the stack and the forest are level (18.9% and 18.4%), and at 1,460 bytes the stack is ahead again (25.5% against 36.0%). For comparison, TANTGen's own padding averages 29 to 32 bytes per packet on portscan both and mirai size, but 246 to 289 bytes on the size variants of ddos, ssdp_flood and portscan, closer to the 512-byte point.

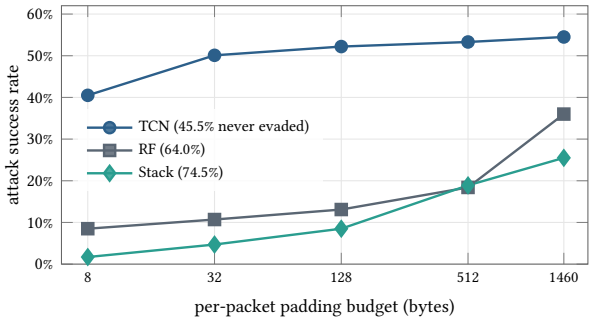


Fig. 3. Attack success of one perturbation crafted with the TCN's gradients and replayed unchanged through each target, at a matched 5% false-positive rate. In parentheses: flows never evaded at any budget.

If the ensemble works as intended, the meta-learner should lean on the forest when the TCN is attacked, and it does. When each base-learner slot of the fusion token is given back its clean value, the share of the change in the verdict due to the forest is 0 without attack, where there is nothing to compensate, and between 0.53 and 0.56 at every non-zero budget. Against the TCN, random search slightly beats PGD at every budget (40.1% against 37.2% at 8 bytes), so in practice the measured attacker is a search attack, and the gradient adds little to it. Both searches are run on the TCN, so the transfer result still holds.

5.4 Purification (RQ2)

Table 5 summarizes the reconstructor. The error on the perturbed dimensions falls 17-fold, and DEPT's own TCN scores purified flows almost exactly like the true clean ones. The main result is measured through the independent NIDS: over 370,700 realized flows, purification raises detection from 0.781 to 0.906, against 0.980 on the clean flows, and so recovers 63.0% of the detection TANTGen had removed. On the eight ML-NIDS sources the rise is from 0.808 to 0.910 against 0.987, an average diluted by the sources where detection was never lost. Of the realizable packets, 97.22% parse

Table 5. Purification on the 384,963 held-out test pairs. The TCN row is circular and kept as a sanity check; compliance counts realizable packets only.

Quantity	Value
MSE on perturbed dimensions (adv. → purified)	0.918 → 0.054 (17.0×)
TCN P(adv) (adv. → purified; clean)	0.675 → 0.428; 0.421
Independent NIDS detection (adv. → purified)	0.781 → 0.906
Clean ceiling of the same NIDS	0.980
Restoration ratio ρ	**63.0**%
`tshark` pass rate	**97.22**%
L7-text packets not realizable (reported apart)	450,758

without errors, and only two sources fall below 0.99: `ddos` (0.84) and `syn_ddos iat` (0.178).

Per source, there are three kinds of result. Where TANTGen's edits were structural, purification removes most of the footprint, and where they had evaded, detection comes back: `ddos size` from 0.005 to 0.817 (clean 0.828), `ddos both` from 0.080 to 0.865, `portscan both` from 0.011 to 0.679 and `portscan size` from 0.013 to 0.479. On `ssdp_flood size` the footprint is also removed (1.000 → 0.003), but no packet of that stream survives L7 realization, so the NIDS cannot score it. On the `both` and `iat` variants of the four reflection floods there is nothing to undo, as the footprint is near zero before purification, and on the `size` variants of `ldap_ddos`, `ntp_ddos` and `ssdp_ddos` the adversarial tokens are bit-identical to the clean ones, because the edit did not happen. Where TANTGen did not evade, purification can lower detection: `dns_ddos` falls from 0.986 to 0.762 in `both`, from 0.993 to 0.756 in `iat` and from 0.986 to 0.765 in `size`, and `portscan iat` from 0.994 to 0.811. The purifier moves these flows towards their clean form, but there the manipulated and clean flows were already close, so it removes information the NIDS was still using. On `dns_ddos size` the footprint falls from 1.000 to 0.302, yet DEPT's own TCN does not see the change, since it cannot tell this source's adversarial and clean flows apart. `mirai`, whose flows are often very short, is where purification works least well: on `size`, the footprint only falls from 1.569 to 0.973.

5.5 End-to-end Deployment and Comparison with a Diffusion Purifier

Deployment. We place the full gated pipeline in front of each protected NIDS: the meta-learner flags flows, only flagged flows are purified, and the rest pass unchanged. For every attack, three realized full streams are scored through its NIDS: the clean attack (the NIDS working as intended), the adversarial traffic (before DEPT) and the defended traffic (after DEPT). Over 2,704,816 scored flows, DEPT cuts attack success from 21.1% to 11.6% and recovers 49.3% of the detection TANTGen had removed (Table 6, left). The gain is large where TANTGen actually evaded: 38 points on `ddos` and 32 on `portscan`, and up to 67 points on single modes (`ddos size` and `portscan both`). Most other attacks barely move, because TANTGen never evaded them. On some, the defense makes things worse: `dns_ddos` loses 22.5 and 22.7 points in `both` and `iat`, `portscan iat` 18.2, `ddos iat` 12.2 and `mirai size` 23.9. On `mirai size`, TANTGen's padding had raised Kitsune's detection from 0.003 to 0.242, and by removing the padding DEPT also removed that gain.

The gate does not prevent these losses. It flags almost all flood traffic, so flows the NIDS was still catching are purified too, except on the `size` variants of the reflection floods, where it flags almost nothing (0.0 to 0.9%), the same blind spot as in Table 4. Purifying every flow gives almost the same detection as the gated pipeline (0.8831 against 0.8836). The gate predicts whether a flow is adversarial, while the pipeline needs to know whether purifying it will help. Because the defended streams also contain flows seen in training, we repeated the measurement on held-out flows only: there the gated pipeline lowers attack success from 31.4% to 17.7%.

Comparison with a published purifier. To compare DEPT with the state of the art, we replace its reconstructor by the diffusion-based purifier of Merzouk et al. [2025], the only network purifier among the surveyed works with public code, and keep the flows, the realization step and the NIDS. The baseline keeps the authors' model and settings and learns from clean traffic only; its amount of noise is chosen as in the original, but on validation flows. Both purifiers are scored on the held-out test pairs (Table 6, right). Attack success is 31.5% without defense, 18.9% with DEPT and 40.3% with the diffusion purifier, and the order is the same on test flows whose original flow appears in no training pair. The setup favours DEPT in two ways: DEPT learns from matched TANTGen and clean flows and is tested against that same attacker, while the baseline sees only clean traffic; and the baseline, designed for flow-level features, had to be adapted to per-packet sizes and timings, and the timing encoding chosen for it explains part of its losses on the floods whose detection depends on timing.

On the five source–mode combinations where TANTGen lowered detection by more than five points (`ddos` and `portscan` in `both` and `size`, and `syn_ddos size`), DEPT recovers 63.5% of the lost detection and the diffusion purifier 0.5%, and no amount of noise did better on the validation flows. A padded or delayed TANTGen flow still looks like realistic traffic, so a purifier that only knows clean traffic has little reason to change it. Elsewhere, both purifiers can lower detection. The diffusion purifier slows down the floods whose detection depends on timing, raising attack success on `syn_ddos` from 4.7% to 39.1%. DEPT is worse than it on the three modes of `dns_ddos`, on `brute_force both` and on `mirai size`, where the diffusion purifier barely changes the traffic.

6 Discussion and Limitations

Research questions. RQ1 is answered positively: fusing a non-differentiable learner with a differentiable one measurably raises the cost of a transferable, gradient-guided evasion, with three limits. The ASR is measured over the flows a target detects without perturbation, and the TCN detects at most 2% of the flows on four of the ten sources, so its side of the comparison rests on a minority of the corpus. On some combinations (`ddos` and `portscan` in `both` and `size`, and `ssdp_flood size`), the forest and the stack are never evaded, which only says that padding and delay cannot overcome that much model confidence. And in practice the measured attacker is a search attack, with PGD only slightly behind it. RQ2 is answered positively under a condition: purification restores detection where the manipulation evaded, and is inert or harmful where it did not.

Table 6. Attack success rate (%) of the protected NIDS per attack, over the three perturbation modes. Left: full realized streams through the gated pipeline, where only flows flagged by the meta-learner are purified; Δ is the reduction in points. Right: held-out test pairs, with DEPT's reconstructor or the diffusion purifier of Merzouk et al. [2025] applied to every flow. Kitsune misses almost all `mirai` traffic even when clean, and 98% of `ssdp_flood`'s packets are dropped (see text), so its rows say little.

		Full streams, gated pipeline				Held-out test pairs, purifier only			
Attack	NIDS	Clean	Before	After	Δ	Clean	No defense	Diffusion	DEPT
`ddos`	ML-NIDS	7.8	61.6	23.4	+38.2	21.0	68.4	89.3	**35.7**
`portscan`	ML-NIDS	0.1	66.1	34.2	+31.9	0.2	66.1	83.6	**34.3**
`syn_ddos`	ML-NIDS	0.1	4.7	0.0	+4.7	0.2	4.7	39.1	**0.0**
`ntp_ddos`	ML-NIDS	0.2	1.1	0.9	+0.2	0.1	1.1	4.4	1.3
`ldap_ddos`	ML-NIDS	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
`ssdp_ddos`	ML-NIDS	0.2	0.2	0.2	0.0	1.4	1.4	1.5	1.5
`brute_force`	ML-NIDS	0.8	0.9	2.1	−1.2	0.6	0.4	0.8	1.6
`dns_ddos`	ML-NIDS	0.6	1.3	19.2	−17.9	1.0	1.3	1.7	24.0
`mirai`	Kitsune	99.7	91.8	99.8	−8.0	99.6	91.3	98.4	99.8
`ssdp_flood`	Kitsune	0.0	0.0	0.0	0.0	99.8	99.9	100.0	99.9
Overall		**1.9**	**21.1**	**11.6**	**+9.5**	**5.7**	**31.5**	**40.3**	**18.9**

Size-perturbed reflection floods. On the `size` variants of the DNS, NTP, SSDP and LDAP floods, every model scores at or below chance, per-attack specialists included. On three of the four, every matched test pair is bit-identical to its clean counterpart, so no detector can tell such a flow from its original; this is a cost of the label convention. On `dns_ddos`, the purifier cuts the footprint to under a third without moving DEPT's verdict, so the cause is likely the representation: its 24 prefix statistics and 1,088 header bits cannot express what separates such a flow from an unedited one. Since these floods supply about a third of the adversarial test flows, most aggregates average near-perfect and below-chance sub-populations.

The gate and the fusion. Purification without an adequate gate subtracts as well as adds, and the gate as built barely gates. The meta-learner, in turn, is beaten by its own Random Forest from packet 2 on, although a stacking ensemble should be no worse than its best base learner; the fusion still provides the packet-1 verdict and the robustness of RQ1.

Validation and scope. The TCN-based purification gain is circular, although no end-to-end claim rests on it. Each attack is judged by a single NIDS, and on `mirai` that NIDS detects almost nothing even on the clean attack. DEPT is compared with one published purifier, adapted to its representation, and only against the attacker it was trained on. The purifier corrects all 1,090 dimensions without being told which the attacker moved, but realization writes back only length and timing, so "attack-agnostic" describes the detector and the reconstructor, not the full deployed pipeline. The features are limited to headers, timing, direction, and payload length and entropy, which remain observable as more traffic is encrypted; payload content is never parsed, which may be where the `dns_ddos` blind spot lies. Finally, tensorization is pure Python, slower than the original C++ nPrint tool, and the latency study that would settle the real-time budget of an inline defense remains to be done.

7 Conclusion and Future Work

DEPT is a streaming defense that detects realizable adversarial manipulation of network traffic from the first packet, and purifies flagged flows back into valid packets before they reach the protected NIDS. Evaluated against a black-box, protocol-compliant generator over ten traffic sources and three perturbation modes, its heterogeneous ensemble makes a gradient-guided evasion measurably more expensive, and its reconstructor, judged by the detector each attack was built to evade, recovers 63% of the lost detection and nearly halves end-to-end attack success, where a published diffusion purifier recovers almost none.

The failures set the future work. A gate trained on whether purification raises or lowers the protected detector's score, which the system already computes, would turn the end-to-end losses into no-ops. Deriving the fields to edit from the residual Δ and checking them against protocol constraints, in the spirit of the SMT-checked perturbations of PANTS [Jin and Apostolaki 2025], would extend realization beyond its two fixed fields. The fusion should never fall below its best base learner, for instance by letting the meta-learner fall back to the stronger opinion when the two diverge. The comparison should be repeated with other purifiers and against generators DEPT was not trained on. And payload content is the first place to look for what separates a size-perturbed DNS flood flow from an unedited one. Beyond these, a static defense is a fixed target: an adversary that probes it long enough can map its blind spots. Casting the defense as a reinforcement learning agent, rewarded by detection outcomes and retrained against an evolving generator such as TANTGen [Nguyen and Reddi 2023; Sutton and Barto 2018], is the direction we consider most important.

Acknowledgments

We thank Inês Pissarra, whose work on TANTGen this work builds upon, for the clarifications and for the materials that made it possible.

References

Elias Abad Rocamora, Grigorios G Chrysos, and Volkan Cevher. 2025. Certified Robustness Under Bounded Levenshtein Distance. In *International Conference on Learning Representations (ICLR)*.

Nour Alhussien and Ahmed Aleroud. 2024. AdvPurRec: Strengthening Network Intrusion Detection with Diffusion Model Reconstruction Against Adversarial Attacks. In *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 1638–1646.

Fahad Alotaibi and Sergio Maffeis. 2024. Mateen: Adaptive ensemble learning for network anomaly detection. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*. 215–234.

Giovanni Apruzzese, Pavel Laskov, and Johannes Schneider. 2023. SoK: Pragmatic assessment of machine learning for network intrusion detection. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 592–614.

Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. 2022. Dos and Don'ts of Machine Learning in Computer Security. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 3971–3988.

Anish Athalye, Nicholas Carlini, and David Wagner. 2018. Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples. In *International Conference on Machine Learning (ICML)*. PMLR, 274–283.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. 2018. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. *arXiv preprint arXiv:1803.01271* (2018).

Battista Biggio, Igino Corona, Giorgio Fumera, Giorgio Giacinto, and Fabio Roli. 2011. Bagging classifiers for fighting poisoning attacks in adversarial classification tasks. In *Multiple Classifier Systems: 10th International Workshop (MCS 2011)*. Springer, 350–359.

Raouf Boutaba, Mohammad A Salahuddin, Noura Limam, Sara Ayoubi, Nashid Shahriar, Felipe Estrada-Solano, and Oscar M Caicedo. 2018. A comprehensive survey on machine learning for networking: evolution, applications and research opportunities. *Journal of Internet Services and Applications* 9, 1 (2018), 16:1–16:99.

Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32.

Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian Goodfellow, Aleksander Madry, and Alexey Kurakin. 2019. *On Evaluating Adversarial Robustness*. Technical Report arXiv:1902.06705. arXiv.

Ruoxin Chen, Zenan Li, Jie Li, Junchi Yan, and Chentao Wu. 2022. On collective robustness of bagging against data poisoning. In *International Conference on Machine Learning*. PMLR, 3299–3319.

Francesco Croce and Matthias Hein. 2020. Reliable Evaluation of Adversarial Robustness with an Ensemble of Diverse Parameter-free Attacks. In *Proceedings of the 37th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 119)*. 2206–2216.

Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572* (2014).

Onat Gungor, Tajana Rosing, and Baris Aksanli. 2022. STEWART: Stacking ensemble for white-box adversarial attacks towards more resilient data-driven predictive maintenance. *Computers in Industry* 140 (2022), 103660.

Mehrdad Hajizadeh, Pegah Golchin, Ehsan Nowroozi, Maria Rigaki, Veronica Valeros, Sebastian García, Mauro Conti, and Thomas Bauschert. 2025. DeepRed: A Deep Learning-Powered Command and Control Framework for Multi-Stage Red Teaming Against ML-based Network Intrusion Detection Systems. In *19th USENIX WOOT Conference on Offensive Technologies (WOOT 25)*. 103–127.

Mohammad J Hashemi, Greg Cusack, and Eric Keller. 2019. Towards evaluation of NIDSs in adversarial setting. In *Proceedings of the 3rd ACM CoNEXT Workshop on Big DAta, Machine Learning and Artificial Intelligence for Data Communication Networks*. 14–21.

Ke He, Dan Dongseong Kim, and Muhammad Rizwan Asghar. 2023. Adversarial machine learning for network intrusion detection systems: A comprehensive survey. *IEEE Communications Surveys & Tutorials* 25, 1 (2023), 538–566.

Jordan Holland, Paul Schmitt, Nick Feamster, and Prateek Mittal. 2021. New Directions in Automated Traffic Analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 3366–3383. doi:10.1145/3460120.3484758

Minhao Jin and Maria Apostolaki. 2025. Robustifying ML-powered Network Classifiers with PANTS. In *34th USENIX Security Symposium (USENIX Security 25)*. 7291–7310.

Pratyay Kumar, Abu Saleh Md Tayeen, Satyajayant Misra, Huiping Cao, Jiefei Liu, Qixu Gong, and Jayashree Harikumar. 2026. NetDiffuser: Deceiving DNN-Based Network Attack Detection Systems with Diffusion-Generated Adversarial Traffic. *arXiv preprint arXiv:2603.08901* (2026).

Vivek Kumar, Kamal Kumar, and Maheep Singh. 2025. NIDS-CBAD: detecting adversarial attacks in network intrusion detection systems using domain constraints. *International Journal of Machine Learning and Cybernetics* 16, 7-8 (2025), 4213–4234.

Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. Towards Deep Learning Models Resistant to Adversarial Attacks. In *6th International Conference on Learning Representations (ICLR)*.

Mohamed Amine Merzouk, Erwan Beurier, Reda Yaich, Nora Boulahia-Cuppens, Frédéric Cuppens, and Foutse Khomh. 2025. Diffusion-based Adversarial Purification for Intrusion Detection. In *Data and Applications Security and Privacy XXXIX: 39th Annual IFIP WG 11.3 Conference (DBSec 2025)*. Springer, 351–370.

Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Network and Distributed System Security Symposium (NDSS)*.

Thanh Thi Nguyen and Vijay Janapa Reddi. 2023. Deep reinforcement learning for cyber security. *IEEE Transactions on Neural Networks and Learning Systems* 34, 8 (2023), 3779–3795.

Maria-Irina Nicolae, Mathieu Sinn, Minh Ngoc Tran, Beat Buesser, Ambrish Rawat, Martin Wistuba, Valentina Zantedeschi, Nathalie Baracaldo, Bryant Chen, Heiko Ludwig, Ian M. Molloy, and Ben Edwards. 2018. *Adversarial Robustness Toolbox v1.0.0*. Technical Report arXiv:1807.01069. arXiv.

Oluseyi Olukola and Nick Rahimi. 2026. AMDS: Attack-Aware Multi-Stage Defense System for Network Intrusion Detection with Two-Stage Adaptive Weight Learning. *arXiv preprint arXiv:2603.00859* (2026).

Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z Berkay Celik, and Ananthram Swami. 2017. Practical black-box attacks against machine learning. In *Proceedings of the 2017 ACM on Asia conference on computer and communications security*. 506–519.

Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32. 8024–8035.

Fabian Pedregosa et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

Fabio Pierazzi, Feargus Pendlebury, Jacopo Cortellazzi, and Lorenzo Cavallaro. 2020. Intriguing properties of adversarial ML attacks in the problem space. In *2020 IEEE symposium on security and privacy (SP)*. IEEE, 1332–1349.

Inês de Almeida Pissarra. 2026. *TANTGen: Transformer-based Adversarial Network Traffic Generator*. Master's thesis. Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal. Unpublished.

Phillip Rieger, Torsten Krauß, Markus Miettinen, Alexandra Dmitrienko, and Ahmad-Reza Sadeghi. 2024. CrowdGuard: Federated Backdoor Detection in Federated Learning. In *Network and Distributed System Security Symposium (NDSS)*.

Benjamin IP Rubinstein, Blaine Nelson, Ling Huang, Anthony D Joseph, Shing-hon Lau, Satish Rao, Nina Taft, and J Doug Tygar. 2009. ANTIDOTE: understanding and defending against poisoning of anomaly detectors. In *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement*. 1–14.

Pouya Samangouei, Maya Kabkab, and Rama Chellappa. 2018. Defense-GAN: Protecting Classifiers Against Adversarial Attacks Using Generative Models. In *International Conference on Learning Representations (ICLR)*.

Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. SciTePress, 108–116. doi:10.5220/0006639801080116

Iman Sharafaldin, Arash Habibi Lashkari, Saqib Hakak, and Ali A. Ghorbani. 2019. Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy. In *2019 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 1–8. doi:10.1109/CCST.2019.8888419

Peishuai Sun, Shuhao Li, Jiang Xie, Hongbo Xu, Zhenyu Cheng, and Rong Yang. 2023. GPMT: Generating practical malicious traffic based on adversarial attacks with little prior knowledge. *Computers & Security* 130 (2023), 103257.

Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (2 ed.). MIT Press, Cambridge, MA.

Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2014. Intriguing properties of neural networks. In *2nd International Conference on Learning Representations (ICLR 2014)*.

Florian Tramèr, Nicholas Carlini, Wieland Brendel, and Aleksander Madry. 2020. On Adaptive Attacks to Adversarial Example Defenses. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 1633–1645.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.

Song Wang, Juan Fernando Balarezo, Sithamparanathan Kandeepan, Akram Al-Hourani, Karina Gomez Chavez, and Benjamin Rubinstein. 2021. Machine learning in network anomaly detection: A survey. *IEEE Access* 9 (2021), 152379–152396.

David H Wolpert. 1992. Stacked generalization. *Neural networks* 5, 2 (1992), 241–259.

Jinzhu Yan, Zhuotao Liu, Yuyang Xie, Shiyu Liang, Lin Liu, and Ke Xu. 2025. CertTA: Certified Robustness Made Practical for Learning-Based Traffic Analysis. In *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 7349–7368.

Zhuolin Yang, Linyi Li, Xiaojun Xu, Shiliang Zuo, Qian Chen, Pan Zhou, Benjamin I. P. Rubinstein, Ce Zhang, and Bo Li. 2021. TRS: Transferability Reduced Ensemble via Promoting Gradient Diversity and Model Smoothness. In *Advances in Neural Information Processing Systems (NeurIPS)*. 17642–17655.

Chaoyun Zhang, Xavier Costa-Pérez, and Paul Patras. 2020. Tiki-Taka: Attacking and defending deep learning-based intrusion detection systems. In *Proceedings of the 2020 ACM SIGSAC conference on cloud computing security workshop*. 27–39.